Ostbayerische Technische Hochschule Regensburg

Fakultät Informatik und Mathematik

Multi-Layer Visualisierung von Tiersichtungsdaten in einer Online-Kartenplattform

Bachelorarbeit

vorgelegt von

Yassir Sbai

Matrikelnummer: 3341177

Betreuer und Prüfer: Prof. Dr. Jan Dünnweber

Regensburg, 3. Oktober 2026

Inhaltsverzeichnis

1 Einleitung **2**

1.1 Motivation und Problemstellung 2

1.2 Zielsetzung der Arbeit 3

1.3 Aufbau der Arbeit 3

2 Grundlagen **5**

2.1 Geodaten und Web-Kartenvisualisierung 5

2.2 Web-Kartenbibliotheken (insb. MapLibre GL JS) 6

3 Analyse der bestehenden Systeme **8**

3.1 Systemübersicht und Komponenten 8

3.2 Simulationsdaten 9

3.2.1 Aufbau des Systems 9

3.2.2 Datenstruktur und Ausgabeformat 10

4 Implementierung **13**

4.1 Umsetzung der Kartenvisualisierung mit MapLibre GL JS 13

4.2 Integration der Teilsysteme 14

4.2.1 Integration Bildextraktionssystem (Anthony Lelchat) 14

4.2.2 Integration Bewegungssimulation (Fabian Breu) 14

4.2.3 Datenbankvereinheitlichung 15

4.3 Neue Datenlayer und Features in der Karte 16

4.4 Interaktions- und Visualisierungskonzepte 17

5 Diskussion **18**

5.1 Bewertung des entwickelten Systems 18

5.2 Limitationen 18

6 Fazit und Ausblick **20**

6.1 Zusammenfassung der Ergebnisse 20

6.2 Ausblick auf zukünftige Erweiterungen 20

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die Erforschung und der Schutz frei lebender Wildtierpopulationen sind zunehmend auf digitale Werkzeuge angewiesen, die räumliche und zeitliche Bewegungsmuster sichtbar und analysierbar machen. Insbesondere bei großen, potenziell konfliktträchtigen Arten wie Bären fallen im Rahmen von Forschungsprojekten heterogene Datenquellen an: GPS-Telemetriedaten frei lebender Tiere, automatisiert erfasste Kamerabilder von Abschreckungssystemen sowie modellbasierte Simulationsdaten zur Vorhersage von Bewegungsmustern. Damit diese Daten für ein Forschungsteam nutzbar werden, müssen sie nicht nur gespeichert, sondern auch verständlich visualisiert werden — im Regelfall auf einer interaktiven Karte, die räumliche Zusammenhänge intuitiv erfassbar macht.

Im Rahmen einer vorausgegangenen Kooperation zwischen der Ostbayerischen Technischen Hochschule (OTH) Regensburg und einer Partnerhochschule wurden im Kontext der Bärenforschung mehrere eigenständige Teilsysteme entwickelt: ein System zur Verwaltung und Kartierung von Bärenabschreckungsmaßnahmen in Japan [1], ein Modell zur Simulation von Bärenbewegungen auf Basis realer GPS-Trajektorien in den Karpaten [2], sowie ein System zur automatisierten Extraktion von Wildtierbildern aus E-Mail-Postfächern [3]. Diese Teilsysteme entstanden im Rahmen separater studentischer Arbeiten, wurden isoliert voneinander implementiert und nutzten für die Kartendarstellung die JavaScript-Bibliothek Leaflet.js in Verbindung mit deren Python-Wrapper Folium.

Mit wachsendem Datenumfang zeigten sich jedoch Grenzen dieses Ansatzes: Folium/Leaflet rendert Geodaten primär über das Document Object Model (DOM) bzw. SVG, was bei einer größeren Anzahl gleichzeitig dargestellter Punkte, Pfade und Flächen zu spürbaren Performanceeinbußen im Browser führt. Zudem existierte für jedes Teilsystem eine eigene Kartenansicht, eigene Filterlogik und teilweise eine eigene Datenhaltung, wodurch eine gemeinsame, systemübergreifende Betrachtung der Daten nicht möglich war. Für ein Forschungsteam, das die Bewegungsmuster realer Tiere mit simulierten Alterna-

tivpfaden und den Standorten von Abschreckungsmaßnahmen in Bezug setzen möchte, stellt diese Fragmentierung eine erhebliche Einschränkung dar.

1.2 Zielsetzung der Arbeit

Ziel dieser Arbeit ist die Konzeption und Implementierung einer einheitlichen, performanten Kartenplattform, die die genannten Teilsysteme in einer gemeinsamen, mehrschichtigen (Multi-Layer) Visualisierung zusammenführt. Dabei sollen mehrere Teilziele erreicht werden:

- **Technologiewechsel der Kartenvisualisierung:** Ablösung der bisherigen Folium/Leaflet-basierten Kartendarstellung durch MapLibre GL JS, eine WebGL-basierte, vektorbasierte Open-Source-Kartenbibliothek, um die Darstellungsperformance bei großen Datenmengen signifikant zu verbessern.
- **Systemintegration:** Zusammenführung der drei bestehenden Teilsysteme — Bärenabschreckung Japan, Bärenbewegungssimulation Karpaten und Bildextraktionssystem — in einer gemeinsamen Anwendung mit einer einzigen, gemeinsam genutzten Kartenkomponente, auf der einzelne Datenebenen unabhängig voneinander ein- und ausgeblendet werden können.
- **Vereinheitlichung der Datenhaltung:** Migration und Konsolidierung der zuvor teils lokal (SQLite, CSV-Dateien) gehaltenen Daten in eine zentrale, kollaborativ nutzbare MariaDB-Datenbank, um konsistenten Mehrbenutzerzugriff und performante serverseitige Filterung zu ermöglichen.
- **Interaktions- und Filterkonzept:** Entwicklung eines nutzerfreundlichen Filter- und Layer-Konzepts, das es Forschenden erlaubt, einzelne Bären, Zeiträume, Saisons und Darstellungsarten auszuwählen, ohne dass dabei die gesamte Anwendung neu geladen werden muss.

Nicht Ziel dieser Arbeit ist die Weiterentwicklung der zugrunde liegenden fachlichen Modelle (z. B. des Bewegungssimulationsmodells) selbst; diese werden als gegeben angenommen und lediglich in die neue Plattform integriert.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich wie folgt: Kapitel 2 legt die technischen und konzeptionellen Grundlagen dar, die für das Verständnis der Arbeit notwendig sind. Dazu zählen die Grundlagen der Geodatenverarbeitung und Web-Kartenvisualisierung sowie

ein Vergleich gängiger Web-Kartenbibliotheken mit besonderem Fokus auf MapLibre GL JS.

Kapitel 3 analysiert die bestehenden Systeme, ihre Architektur, ihre Datenstrukturen sowie die Simulationsdaten, die in die neue Plattform integriert werden sollen, und leitet daraus die Anforderungen an das zu entwickelnde System ab.

Kapitel 4 beschreibt die eigentliche Implementierung: die Umsetzung der Kartenvisualisierung mit MapLibre GL JS, die Integration der einzelnen Teilsysteme, die Vereinheitlichung der Datenbank sowie die entwickelten Interaktions- und Visualisierungskonzepte.

Kapitel 5 diskutiert und bewertet das entwickelte System kritisch und ordnet dessen Grenzen ein. Kapitel 6 fasst die Ergebnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Erweiterungen des Systems.

Kapitel 2

Grundlagen

2.1 Geodaten und Web-Kartenvisualisierung

Geodaten sind Daten, die einen direkten Raumbezug besitzen, also einer bestimmten Position auf der Erdoberfläche zugeordnet werden können [4]. Man unterscheidet grundsätzlich zwischen Vektordaten und Rasterdaten. Vektordaten beschreiben geometrische Objekte in Form von Punkten, Linien und Polygonen, die jeweils durch Koordinatenlisten definiert sind — beispielsweise ein einzelner GPS-Messpunkt eines besenderten Bären (Punkt), dessen zurückgelegter Bewegungspfad (Linie) oder ein abgegrenztes Untersuchungsgebiet (Polygon). Rasterdaten hingegen bestehen aus einem regelmäßigen Gitter von Zellen mit zugeordneten Werten, wie es etwa bei Luftbildern, Höhenmodellen oder klassischen Kartenkacheln (Tiles) der Fall ist.

Um Geodaten eindeutig zu positionieren, werden Koordinatenreferenzsysteme (Coordinate Reference Systems, CRS) verwendet. Für die Web-Kartendarstellung hat sich das World Geodetic System 1984 (WGS84, EPSG:4326) als De-facto-Standard etabliert, das Positionen als geografische Breite (Latitude) und Länge (Longitude) angibt. Für die Kachelbasierte Darstellung im Web wird davon meist in die sogenannte Web-Mercator-Projektion (EPSG:3857) transformiert, welche die Kugelform der Erde auf eine ebene Fläche abbildet und dabei Winkeltreue zulasten der Flächentreue bevorzugt — ein für die Navigation und Web-Kartendienste akzeptabler Kompromiss [5].

Für den Datenaustausch zwischen Server und Client hat sich im Web-Kontext das Format GeoJSON durchgesetzt, eine auf JSON basierende offene Spezifikation zur Kodierung geographischer Datenstrukturen [6]. Eine GeoJSON-`FeatureCollection` besteht aus einer Liste von `Feature`-Objekten, die jeweils eine Geometrie (z. B. `Point`, `LineString`, `Polygon`) und beliebige zusätzliche Eigenschaften (`properties`) in Form von Schlüssel-Wert-Paaren enthalten können. Dieses Format eignet sich besonders gut für die Übertragung dynamisch generierter Kartendaten aus einer Backend-Anwendung an eine im Browser laufende Kartenkomponente, da es sowohl von serverseitigen Programmiersprachen wie

Python als auch nativ von JavaScript-Kartenbibliotheken verarbeitet werden kann.

Web-basierte Kartenanwendungen folgen üblicherweise einer Client-Server-Architektur: Der Server hält die Rohdaten (z. B. in einer relationalen oder Geodatenbank) vor, bereitet sie auf Anfrage auf und liefert sie in einem für den Client verarbeitbaren Format aus; der Client — im Regelfall ein Webbrowser — übernimmt das eigentliche Rendering der Karte sowie die Interaktion mit dem Nutzer. Historisch wurden Kartenbilder häufig vollständig serverseitig gerendert und als fertige Rasterbilder (Kartenkacheln) an den Client übertragen. Mit der Verbreitung leistungsfähiger Grafik-APIs im Browser, insbesondere WebGL, hat sich der Trend zunehmend in Richtung client-seitigem Rendering von Vektordaten verschoben: Anstelle vorgerenderter Bilder werden Vektorkacheln (Vector Tiles) übertragen, deren visuelle Darstellung (Farben, Beschriftung, Symbolik) erst im Browser anhand eines deklarativen Style-Dokuments berechnet wird. Dieser Ansatz reduziert die übertragene Datenmenge, ermöglicht eine flüssige, GPU-beschleunigte Interaktion (Zoomen, Rotieren) und erlaubt es, denselben Datenbestand dynamisch unterschiedlich zu stylen, ohne serverseitig neue Bilder generieren zu müssen [7].

2.2 Web-Kartenbibliotheken (insb. MapLibre GL JS)

Zur Einbettung interaktiver Karten in Webanwendungen existiert eine Reihe etablierter JavaScript-Bibliotheken, die sich in ihrem Rendering-Ansatz, ihrem Funktionsumfang und ihrer Lizenzierung unterscheiden.

Leaflet.js ist eine seit 2011 verbreitete, leichtgewichtige Open-Source-Bibliothek, die Karten primär über gekachelte Rasterbilder darstellt und einzelne Vektorobjekte (Marker, Linien, Polygone) über SVG- bzw. Canvas-Elemente im DOM rendert [8]. Aufgrund ihres einfachen API und der geringen Einstiegshürde wird Leaflet häufig in Kombination mit dem Python-Paket Folium eingesetzt, das eine deklarative Python-Schnittstelle zur Erzeugung von Leaflet-Karten bereitstellt und insbesondere im Data-Science- und Prototyping-Umfeld verbreitet ist. Der DOM-basierte Rendering-Ansatz stößt jedoch bei einer großen Anzahl gleichzeitig dargestellter Objekte (mehrere tausend Punkte oder komplexe Liniengeometrien) an Performancegrenzen, da jedes Objekt ein eigenes DOM- bzw. SVG-Element erzeugt, dessen Verwaltung und Neuzeichnung mit wachsender Objektzahl zunehmend rechenintensiv wird.

Mapbox GL JS führte demgegenüber ab 2014 einen WebGL-basierten Rendering-Ansatz ein, bei dem Vektorkacheln direkt auf der Grafikkarte gerendert werden, was eine deutlich höhere Darstellungsleistung bei großen Datenmengen sowie flüssige 3D-Perspektiven und Rotationen ermöglicht [9]. Mit der Änderung der Lizenzbedingungen von Mapbox GL JS im Jahr 2020 hin zu einer proprietären Lizenz entstand als Community-Fork der letzten frei lizenzierten Version das Projekt **MapLibre GL JS**, das seither unter der offenen BSD-3-Clause-Lizenz weiterentwickelt wird [10]. MapLibre GL JS übernimmt

dabei die WebGL-basierte Rendering-Architektur sowie die deklarative Style-Spezifikation von Mapbox GL JS, bleibt jedoch vollständig quelloffen und ist nicht an einen kommerziellen Kartenanbieter gebunden.

Zentrales Konzept von MapLibre GL JS ist die Trennung von Daten (Sources) und deren visueller Darstellung (Layers). Eine `Source` referenziert einen Datensatz — etwa eine GeoJSON-`FeatureCollection` oder einen Vector-Tile-Endpunkt — während ein oder mehrere `Layer` festlegen, wie die Daten dieser Quelle dargestellt werden (z. B. als Kreis mit bestimmter Farbe und Radius, als Linie mit bestimmter Strichstärke, oder als Flächenfüllung). Über sogenannte Paint- und Layout-Properties lassen sich diese Darstellungseigenschaften dynamisch anhand von Feature-Eigenschaften steuern, etwa um GPS-Punkte eines Simulationsmodells farblich nach ihrem Typ (realer Messpunkt vs. simulierte Alternative) zu unterscheiden. Die gesamte Kartenkonfiguration — Basiskarte, Datenquellen, Layer und deren Styling — wird dabei über ein deklaratives JSON-Dokument nach der Mapbox-Style-Spezifikation beschrieben, was eine hohe Flexibilität bei gleichzeitig guter Nachvollziehbarkeit der Kartendefinition ermöglicht [11].

Für die vorliegende Arbeit ist MapLibre GL JS aus mehreren Gründen die geeignete Wahl: Erstens ermöglicht der WebGL-basierte Rendering-Ansatz eine performante Darstellung auch größerer Punkt- und Pfadmengen, wie sie bei der gleichzeitigen Visualisierung realer GPS-Trajektorien, simulierter Alternativpfade und Gerätestandorte anfallen. Zweitens erlaubt die freie, quelloffene Lizenzierung den uneingeschränkten Einsatz im Forschungskontext ohne Abhängigkeit von einem kommerziellen Kartenanbieter oder API-Kosten. Drittens lässt sich MapLibre GL JS, da es sich um eine reine JavaScript-Bibliothek handelt, über eine HTML/JavaScript-Einbettung unmittelbar in eine Python-basierte Webanwendung wie Streamlit integrieren, ohne auf ein spezialisiertes, aber möglicherweise weniger flexibles Python-Wrapper-Paket angewiesen zu sein.

Kapitel 3

Analyse der bestehenden Systeme

3.1 Systemübersicht und Komponenten

Das im Rahmen dieser Arbeit weiterentwickelte Gesamtsystem setzt sich aus vier fachlich unterschiedlichen, historisch getrennt entwickelten Komponenten zusammen, die im Folgenden kurz charakterisiert werden. Tabelle 3.1 gibt einen Überblick.

Tabelle 3.1: Übersicht der in die Plattform integrierten Teilsysteme

Teilsystem	**Ursprung**	**Datenquelle**	**Zweck**
Bärenabschreckung Japan	Vorarbeit [1]	Geräte-Standorte, Kamerabilder, Freihand-Polygone (DB)	Kartierung und Verwaltung von Abschreckungsmaßnahmen
Bärenbewegungssimulation	Vorarbeit [2]	GPS-Trajektorien Karpaten (SQLite), Geschwindigkeitsstatistiken	Modellbasierte Generierung alternativer Bewegungsschritte
Bildextraktion aus E-Mails	Vorarbeit [3]	Postfach via Microsoft Graph API	Automatisierte Erfassung neuer Kamerabilder
Carpathian-Bären-Tracking	Bestandteil dieser Arbeit erweitert	GPS-Tracking-Daten (MariaDB, CSV-Fallback)	Darstellung realer Bewegungspfade rumänischer Braunbären

Alle vier Komponenten wurden vor Beginn dieser Arbeit unabhängig voneinander implementiert und verfügten jeweils über eigene, isolierte Datenhaltung sowie eigene Kartenansichten auf Basis von Folium/Leaflet. Eine gemeinsame Nutzung war allenfalls durch manuelles Wechseln zwischen mehreren Anwendungen möglich. Als gemeinsamer Nenner dient in der bestehenden Architektur die Klasse `WildlifeDatabase` (Datei `wildlife_db_mariadb.py`), welche die Verbindung zu einer zentralen MariaDB-Instanz

(Datenbank `ursus`) kapselt und generische Methoden zur Ausführung parametrisierter SQL-Abfragen (`execute_query`, `execute_update`) bereitstellt. Diese Datenzugriffsschicht wird von mehreren Teilsystemen wiederverwendet, war zu Beginn dieser Arbeit jedoch inkonsistent eingebunden: Während die Bildextraktion und die Japan-Komponente bereits vollständig auf MariaDB umgestellt waren, griff die Carpathian-Komponente wahlweise auf die zentrale Datenbank oder, als Fallback bei fehlenden Daten, auf lokale CSV-Dateien zurück, und die Bewegungssimulation nutzte weiterhin eine separate, lokal abgelegte SQLite-Datenbank.

Abbildung 3.1 skizziert die Zielarchitektur, wie sie im Rahmen dieser Arbeit umgesetzt wurde: Eine einzelne Streamlit-Anwendung (`map.py`) fungiert als zentraler Einstiegspunkt, lädt die Daten der einzelnen Teilsysteme parallel in der Seitenleiste konfigurierbar vor und übergibt sie an eine gemeinsame Kartenkomponente (`maplibre_component.py`), die alle Layer in einer einzigen MapLibre-GL-Instanz zusammenführt.

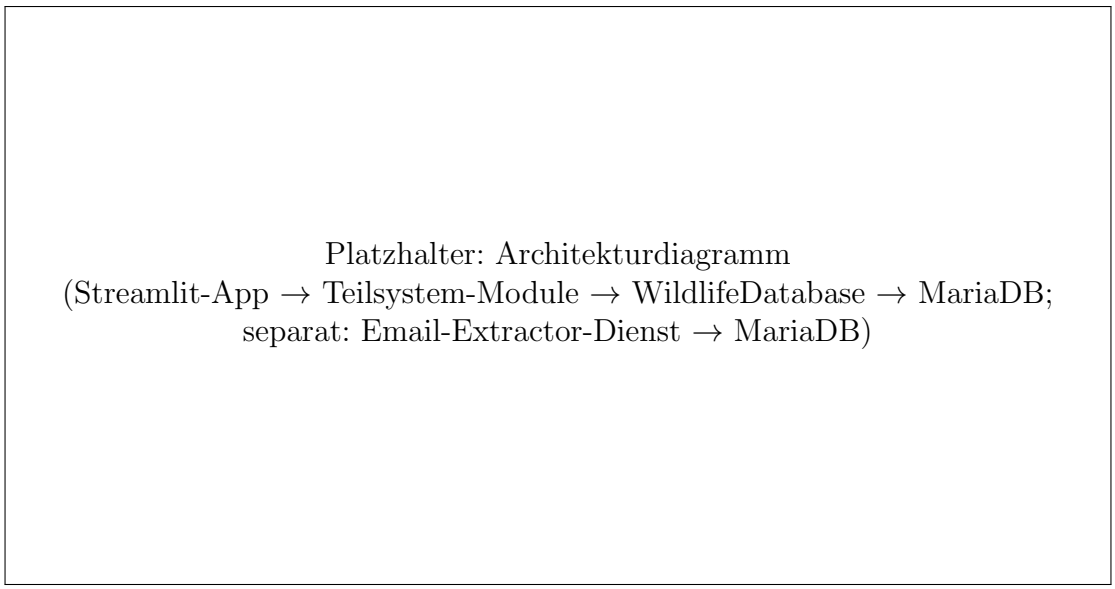


Abbildung 3.1: Zielarchitektur: gemeinsame Datenzugriffsschicht und gemeinsame Kartenkomponente

3.2 Simulationsdaten

3.2.1 Aufbau des Systems

Die Bewegungssimulation basiert auf einem sogenannten Step-Selection-Function-Modell (SSF), einem in der Bewegungsökologie etablierten statistischen Verfahren zur Analyse und Vorhersage von Tierbewegungen [12]. Grundidee eines SSF-Modells ist es, für jeden real beobachteten Bewegungsschritt eines Tieres eine Menge plausibler, aber nicht tatsächlich eingeschlagener Alternativschritte zu generieren. Durch den statistischen Vergleich der Eigenschaften (z. B. Landbedeckung, Entfernung zu Straßen) des realen Schritts

gegenüber denen der Alternativschritte lässt sich modellieren, welche Umgebungsfaktoren die Bewegungsentscheidung eines Tieres beeinflussen.

Im bestehenden System ist dieses Modell in der Klasse `StepSelectionModel` implementiert, die auf einer Reihe von Hilfsklassen aufbaut (u. a. `StepTrajectorySet`, `TrajectorySegment`, `TrajectoryPoint`, `SeasonalTrajectoryProcessor`). Die realen GPS-Trajektorien rumänischer Braunbären werden dabei in saisonale Segmente unterteilt (Frühling, Sommer, Herbst, Winter), da sich das Bewegungsverhalten von Bären je nach Jahreszeit deutlich unterscheidet, u. a. bedingt durch die herbstliche Hyperphagie-Phase oder die winterliche Ruhephase. Für jedes so entstandene Segment wird zusätzlich auf eine Tabelle geschwindigkeitsbezogener Referenzwerte (`bear_speeds`) zugegriffen, die nach Geschlecht, Altersklasse, Saison, Tageszeit und Datenquelle differenziert vorliegt und zur Modellierung realistischer Schrittlängen herangezogen wird.

Im Rahmen dieser Arbeit wurde das Simulationsmodell, das ursprünglich als eigenständiges, lokal ausgeführtes Skript konzipiert war, in eine wiederverwendbare Integrationsschicht (`bear_simulation/__init__.py`) überführt. Diese Schicht lädt das Modell beim ersten Zugriff einmalig und hält es über die gesamte Nutzersitzung im Arbeitsspeicher (`st.cache_resource`), anstatt es bei jeder Interaktion neu zu initialisieren. Dabei wurde zusätzlich ein Kompatibilitätsproblem gelöst: Die ursprüngliche Implementierung griff bei Bedarf direkt auf eine SQLite-Verbindung zu, um Geschwindigkeitswerte nachzuschlagen. Da Streamlit eingehende Anfragen jedoch nicht zwingend im selben Thread bearbeitet, in dem die Datenbankverbindung ursprünglich geöffnet wurde, SQLite-Verbindungsobjekte in Python jedoch nicht threadübergreifend verwendet werden dürfen, wurden die relevanten Geschwindigkeitswerte beim einmaligen Laden vollständig in ein In-Memory-Dictionary überführt; alle nachfolgenden Zugriffe erfolgen ausschließlich über dieses Dictionary.

3.2.2 Datenstruktur und Ausgabeformat

Für die Einbindung in die gemeinsame Kartenkomponente wurde eine Konvertierungsfunktion `get_simulation_geojson(bear_id, season, num_alternatives)` implementiert, die das Ausgabeformat des Simulationsmodells in eine für MapLibre GL JS konsumierbare GeoJSON-Struktur überführt. Das Ergebnis besteht aus zwei `FeatureCollections`: `points` enthält sowohl die realen GPS-Messpunkte als auch die dazugehörigen, modellgenerierten Alternativpunkte je Schritt; `paths` enthält den aus den realen Punkten gebildeten Bewegungspfad als `LineString`. Tabelle 3.2 listet die wichtigsten Eigenschaften eines Punkt-Features auf.

Listing 3.1 zeigt beispielhaft die Struktur eines vereinfachten Ausschnitts der resultierenden GeoJSON-Ausgabe.

Listing 3.1: Vereinfachtes Beispiel eines Punkt-Features der Simulations-GeoJSON

```
1 {
```

Tabelle 3.2: Feature-Eigenschaften der Simulations-GeoJSON (Ausschnitt)

Property	**Typ**	**Bedeutung**
`bear_id`	String	ID des Bären, dem der Schritt zugeordnet ist
`type`	String	`"real"` für gemessene Schritte, `älternative"` für simulierte Schritte
`step_idx`	Integer	Laufender Index des Schritts innerhalb des Segments
`angle`	Float	Richtungswinkel des Schritts (Grad)
`length_m`	Float	Schrittlänge in Metern
`color`	String	Für das Rendering vorbestimmter Farbwert (blau = real, orange = alternativ)

```
  "type": "Feature",
  "properties": {
    "bear_id": "10",
    "type": "alternative",
    "step_idx": 4,
    "angle": 132.6,
    "length_m": 812.3,
    "color": "#ff9800"
  },
  "geometry": {
    "type": "Point",
    "coordinates": [25.041, 45.663]
  }
}
```

Die Anzahl der pro realem Schritt generierten Alternativschritte ist über den Parameter `num_alternatives` konfigurierbar und wird über die Nutzeroberfläche im Bereich 3 bis 20 einstellbar gemacht (siehe Abschnitt 4.4). Bei n realen Schritten und k Alternativen je Schritt ergibt sich die Gesamtzahl der dargestellten Simulationspunkte zu

$$N_{\text{Punkte}} = n \cdot (k + 1), \tag{3.1}$$

da neben den k Alternativen je Schritt stets auch der reale Messpunkt selbst mit dargestellt wird. Diese Beziehung ist insbesondere für die Einschätzung der Rendering-Last relevant, da bereits bei moderater Schrittzahl und einer für aussagekräftige Modellvergleiche sinnvollen Alternativenzahl schnell mehrere tausend Punkte gleichzeitig dargestellt werden müssen — ein Szenario, das die in Abschnitt **??** beschriebenen Grenzen des DOM-basierten Renderings deutlich überschreitet und den in dieser Arbeit vorgenomme-

nen Wechsel zu MapLibre GL JS zusätzlich motiviert.

Kapitel 4

Implementierung

4.1 Umsetzung der Kartenvisualisierung mit MapLibre GL JS

Die gemeinsame Kartenkomponente ist in der Datei `maplibre_component.py` implementiert und folgt dem Prinzip, eine vollständige, in sich geschlossene HTML/JavaScript-Seite zu erzeugen, die über die Streamlit-Funktion `components.html()` als eingebettetes `iframe` in die Anwendung eingebunden wird. Diese Vorgehensweise wurde gewählt, da MapLibre GL JS als reine JavaScript-Bibliothek ausgeliefert wird und Streamlit selbst über kein natives Kartenwidget mit vergleichbarem Funktionsumfang verfügt.

Die zentrale Funktion `render_maplibre()` nimmt die aus den Teilsystemen aufbereiteten GeoJSON-Daten entgegen, serialisiert sie als JSON in den generierten HTML-Quelltext und initialisiert innerhalb des eingebetteten JavaScript-Codes eine einzelne `maplibregl.Map`-Instanz. Für jede Datenquelle wird beim Laden der Karte eine `Source` sowie ein oder mehrere zugehörige `Layer` hinzugefügt (Funktion `addSourcesAndLayers`); die Hilfsfunktion `addLayerSafe` stellt dabei sicher, dass Layer idempotent, also nicht mehrfach, hinzugefügt werden. Damit die von der Basiskarte gerenderten Beschriftungen (z. B. Städte- und Ländernamen) weiterhin über den eigenen Datenlayern sichtbar bleiben, wird jeder neue Layer gezielt vor dem ersten in der Basiskarte vorkommenden Symbol-Layer eingefügt (Funktion `firstSymbolLayerId`).

Ein zentrales Gestaltungsmerkmal der Implementierung ist die strikte Trennung zwischen serverseitig (Streamlit/Python) und clientseitig (JavaScript im Browser) ausgelösten Interaktionen. Das Ein- und Ausblenden einzelner Layer über die auf der Karte platzierten Checkboxen (siehe Tabelle 4.1) wird ausschließlich clientseitig über `map.setLayoutProperty(layer, "visibility", ...)` realisiert und löst keinen erneuten Rundlauf zum Streamlit-Server aus. Dies ist eine bewusste Entwurfsentscheidung: Ein erneuter Server-Roundtrip würde einen vollständigen Neuaufbau der Streamlit-Seite und damit ein erneutes Rendering der gesamten `iframe`-Karte samt Neuanfrage der Kartenkacheln erzwingen, was sich in der

Praxis als spürbare Verzögerung bemerkbar macht.

Tabelle 4.1: Auf der Karte verfügbare Basiskartenstile

Stilname	**Typ**
Liberty	Vektorkachel (OpenFreeMap)
Bright	Vektorkachel (OpenFreeMap)
Positron	Vektorkachel (CARTO)
Dark	Vektorkachel (CARTO)
OSM Raster	Rasterkachel (OpenStreetMap)
Esri Satellite	Rasterkachel (Esri World Imagery)

Beim Wechsel des Basiskartenstils wird die gesamte Stildefinition über `map.setStyle()` ausgetauscht. Da dieser Vorgang sämtliche zuvor hinzugefügten Quellen und Layer entfernt, werden diese nach Abschluss des Stilwechsels über den Event-Listener `map.once('style.load', ...)` automatisch wiederhergestellt; die Option `diff:false` erzwingt dabei einen vollständigen Stil-Reset, um inkonsistente Zwischenzustände zu vermeiden.

4.2 Integration der Teilsysteme

4.2.1 Integration Bildextraktionssystem (Anthony Lelchat)

Das Bildextraktionssystem läuft als eigenständiger, von der Streamlit-Anwendung unabhängiger Prozess (`email_extractor`), der über die Microsoft Graph API periodisch ein E-Mail-Postfach nach neu eingegangenen Wildtierbildern durchsucht (Klasse `MicrosoftGraphApi`), diese extrahiert (Klasse `WildlifePictureExtractor`) und über eine eigene Datenzugriffsschicht (`DBManager`) in die zentrale MariaDB-Datenbank schreibt. Die Kopplung an die Kartenanwendung erfolgt lose über die gemeinsam genutzte Datenbank: Die Streamlit-Anwendung liest die vom Extraktionsprozess eingetragenen Bilddaten, ohne direkt mit diesem zu kommunizieren.

Um Nutzenden dennoch sichtbar zu machen, wenn neue Bilder eingetroffen sind, wurde im Rahmen dieser Arbeit ein Status-Widget (`status_widget.py` in Verbindung mit `new_images_checker.py`) ergänzt, das beim Laden der Anwendung den Zeitpunkt der letzten Prüfung persistent in der Datenbank hinterlegt und alle seit diesem Zeitpunkt neu hinzugekommenen Bilder abfragt und in der Seitenleiste anzeigt.

4.2.2 Integration Bewegungssimulation (Fabian Breu)

Wie in Abschnitt 3.2 dargestellt, wurde die Bewegungssimulation über die neu geschaffene Integrationsschicht `bear_simulation/__init__.py` an die gemeinsame Kartenanwendung angebunden. Die Funktionen `get_simulation_options()` und `get_simulation_geojson()`

bilden dabei die öffentliche Schnittstelle, über die `map.py` sämtliche für die Filterung und Darstellung benötigten Informationen bezieht, ohne Kenntnis über die interne Funktionsweise des Simulationsmodells haben zu müssen.

Im Verlauf der Implementierung zeigte sich, dass die Neuberechnung der Alternativschritte (`generate_alternative_steps`) bei jedem erneuten Durchlauf des Streamlit-Skripts ausgeführt wurde — also auch dann, wenn ein Nutzer lediglich einen fachlich nicht verwandten Filter (z. B. der Carpathian-Komponente) änderte. Da Streamlit bei jeder Nutzerinteraktion das gesamte Anwendungsskript von oben nach unten neu ausführt, führte dies zu spürbaren Wartezeiten. Als Lösung wurde die Funktion `get_simulation_geojson` mit dem Streamlit-Dekorator `@st.cache_data` versehen, der das Ergebnis für eine gegebene Kombination aus Bären-ID, Saison und Alternativenzahl zwischenspeichert und bei unveränderten Parametern ohne erneute Modellberechnung zurückgibt. Dieselbe Optimierung wurde für die GeoJSON-Erzeugung der Carpathian-Komponente (`get_carpathian_geojson`) vorgenommen, die zuvor bei jedem Skriptdurchlauf erneut eine Datenbankabfrage sowie eine Neuberechnung des Downsamplings auslöste.

4.2.3 Datenbankvereinheitlichung

Zu Beginn dieser Arbeit lag der Bilddatenbestand in zwei getrennten Tabellenschemata vor, die im Zuge der Weiterentwicklung der einzelnen Teilsysteme unabhängig voneinander entstanden waren. Um eine einheitliche, redundanzfreie Abfrage über alle Bildquellen hinweg zu ermöglichen, wurde im Rahmen dieser Arbeit eine vereinheitlichte Tabelle `bears_images_unified` eingeführt. Die Migration erfolgte in drei Schritten, die jeweils in einem eigenständigen, einmalig auszuführenden Skript festgehalten sind:

1. `Create_status_table.py` — Anlage einer Tabelle `app_status` zur Nachverfolgung von Anwendungszuständen (u. a. Zeitpunkt der letzten Prüfung auf neue Bilder).

2. `create_unified_table.py` — Anlage des neuen, vereinheitlichten Tabellenschemas `bears_images_unified`.

3. `migrate_to_unified_table.py` — Übertragung des Bestands aus den bisherigen Quellen (Dateisystem-basierte Ablage sowie bestehende `wildlife_images`-Tabelle) in das neue Schema, unter Beibehaltung der ursprünglichen Tabellen zur Absicherung gegen Datenverlust.

Dieses Vorgehen folgt dem Prinzip der additiven, nicht destruktiven Migration: Bestehende Tabellen wurden nicht gelöscht oder überschrieben, sondern lediglich um eine konsolidierte Sicht ergänzt, sodass im Fehlerfall jederzeit auf den ursprünglichen Datenbestand zurückgegriffen werden konnte.

Ergänzend wurde im Rahmen dieser Arbeit die Fehlerbehandlung der zentralen Datenzugriffsschicht überarbeitet. In der ursprünglichen Implementierung wurden Datenbankfehler (z. B. bei nicht erreichbarem Server) innerhalb der Methoden `execute_query`, `get_deterrent_devices`, `get_markers` und `get_polygons` vollständig abgefangen und lediglich auf der Konsole protokolliert; der jeweilige Aufrufer erhielt in diesem Fall kommentarlos eine leere Ergebnismenge zurück. In der Praxis erschwerte dieses Verhalten die Fehlerdiagnose erheblich, da eine leere Datenbank und eine nicht erreichbare Datenbank aus Sicht der Anwendung ununterscheidbar waren. Im Rahmen dieser Arbeit wurde dieses Verhalten dahingehend angepasst, dass Verbindungsfehler nicht mehr stillschweigend verschluckt, sondern an die aufrufende Ebene weitergereicht werden, wo sie dem Nutzer als konkrete Fehlermeldung angezeigt werden. Zusätzlich wurde ein expliziter Verbindungs-Timeout konfiguriert, um bei nicht erreichbarem Datenbankserver ein zügiges Fehlschlagen anstelle eines minutenlangen Wartens auf Betriebssystemebene zu erzwingen.

4.3 Neue Datenlayer und Features in der Karte

Über die reine Zusammenführung bestehender Layer hinaus wurden im Rahmen dieser Arbeit mehrere Darstellungsmerkmale neu eingeführt bzw. erweitert:

- **Differenzierte Punktdarstellung der Simulation:** Reale GPS-Punkte und simulierte Alternativschritte werden farblich (blau vs. orange) sowie über Radius und Deckkraft visuell unterschieden, sodass auf einen Blick erkennbar ist, welcher Anteil der dargestellten Punkte auf tatsächlichen Messungen beruht.
- **Kontextbezogene Popups:** Ein Klick auf ein Feature öffnet ein layerspezifisches Popup mit den relevanten Metadaten (z. B. Bild-Anzahl je Abschreckungsgerät, Zeitstempel und Saison eines GPS-Punkts, Schrittlänge und Richtungswinkel eines Simulationsschritts).
- **Automatisches Zoomen beim Einblenden eines Layers:** Wird ein Layer über die Checkbox aktiviert, berechnet die Karte automatisch die Begrenzungsbox (*Bounding Box*) der zugehörigen Daten und fährt die Ansicht mittels `fitBounds` bzw. `flyTo` entsprechend heran, sodass Nutzer die neu eingeblendeten Daten nicht manuell suchen müssen.
- **Statusanzeige neu eingetroffener Bilder:** Wie in Abschnitt 4.2.1 beschrieben, informiert ein Widget in der Seitenleiste über neu hinzugekommene Bilder seit der letzten Prüfung.

4.4 Interaktions- und Visualisierungskonzepte

Die Filterung der Carpathian- und Simulationsdaten (Bärenauswahl, Zeitraum, Saison, Anzahl der Alternativschritte) erfolgt jeweils über ein Streamlit-`form`-Element. Diese Entwurfsentscheidung ist unmittelbar mit dem in Abschnitt 4.2.2 beschriebenen Performance-Problem verknüpft: Ohne Gruppierung in einem `form`-Element würde bereits jede einzelne Änderung eines Filterwidgets (z. B. das Verschieben eines Schiebereglers) einen vollständigen Neudurchlauf des Anwendungsskripts samt Neuberechnung aller abhängigen Daten auslösen. Durch die Bündelung mehrerer Eingabefelder in einem Formular mit explizitem Bestätigungs-Button (Änwenden") wird die serverseitige Neuberechnung auf den tatsächlichen Bestätigungszeitpunkt beschränkt.

Insgesamt verfolgt die Anwendung damit ein zweistufiges Interaktionskonzept: Fachliche Filterentscheidungen (welche Daten werden geladen) werden serverseitig über Streamlit-Formulare mit expliziter Bestätigung verarbeitet, während rein darstellungsbezogene Interaktionen (welche bereits geladenen Daten sind sichtbar, welcher Kartenausschnitt wird angezeigt) vollständig clientseitig in JavaScript ohne Server-Interaktion erfolgen. Diese Trennung reduziert die Anzahl der notwendigen Server-Roundtrips auf ein Minimum und war eine zentrale Voraussetzung, um die in Kapitel 5 diskutierte gefühlte Reaktionsgeschwindigkeit der Anwendung zu erreichen.

Kapitel 5

Diskussion

5.1 Bewertung des entwickelten Systems

Die im Rahmen dieser Arbeit umgesetzte Zusammenführung der drei Teilsysteme in eine gemeinsame, MapLibre-GL-basierte Kartenanwendung erfüllt die in Abschnitt 1.2 formulierten Ziele in weiten Teilen. Durch den Wechsel von DOM-basiertem Rendering (Folium/Leaflet) zu WebGL-basiertem Rendering (MapLibre GL JS) lassen sich auch größere Datenmengen — insbesondere die gemäß Gleichung (3.1) schnell in den Tausenderbereich wachsende Punktzahl der Bewegungssimulation — spürbar flüssiger darstellen als in der ursprünglichen Implementierung.

Die Vereinheitlichung der Datenzugriffsschicht sowie die Einführung von serverseitigem Caching (`st.cache_data`, `st.cache_resource`) haben zudem die wahrgenommene Reaktionsgeschwindigkeit der Anwendung deutlich verbessert: Wiederholte Interaktionen mit unveränderten Filterparametern werden nun aus dem Cache bedient, anstatt die zugrunde liegenden — teils rechenintensiven — Berechnungen (Modellsimulation, Datenbankabfragen, Downsampling) erneut auszuführen.

5.2 Limitationen

Trotz der erzielten Verbesserungen weist das System mehrere grundsätzliche Limitationen auf, die im Rahmen dieser Arbeit bewusst nicht adressiert wurden bzw. außerhalb des Handlungsspielraums der Anwendung selbst liegen:

- **Zentraler Single Point of Failure:** Sämtliche Teilsysteme mit Ausnahme der Bewegungssimulation sind vollständig von der Erreichbarkeit einer einzelnen, zentral gehosteten MariaDB-Instanz abhängig. Fällt dieser Server aus oder ist er — etwa aufgrund von Netzwerk- oder VPN-Restriktionen — nicht erreichbar, stehen den betroffenen Teilsystemen keinerlei Daten zur Verfügung. Im Rahmen der praktischen Erprobung dieser Arbeit trat dieser Fall konkret auf und unterstreicht, dass für

einen produktiven Mehrnutzerbetrieb Redundanz- bzw. Monitoring-Konzepte für die zentrale Datenbank sinnvoll wären, die jedoch außerhalb des Aufgabenbereichs dieser Arbeit liegen.

- **Keine Echtzeit-Kollaboration:** Wie bereits in der Vorarbeit zum Japan-Teilsystem festgestellt [1], verhindert das System nicht, dass mehrere Nutzer gleichzeitig widersprüchliche Änderungen an denselben Daten vornehmen. Da im Rahmen dieser Arbeit primär die Kartenvisualisierung und weniger die Schreiblogik im Fokus stand, wurde dieses Problem nicht adressiert.

- **Eingebettete iframe-Architektur statt natives Streamlit-Component:** Die Einbindung von MapLibre GL JS erfolgt über eine vollständig serverseitig gerenderte HTML-Seite, die bei jedem Neudurchlauf des Streamlit-Skripts erneut als Ganzes an den Client übertragen und im Browser neu initialisiert wird. Eine tiefere Integration über ein bidirektionales Streamlit-Custom-Component (mit inkrementeller Datenübertragung) wäre technisch eleganter, hätte jedoch den Implementierungsaufwand dieser Arbeit deutlich überschritten.

- **Hartkodierte Konfiguration:** Verbindungsparameter zur zentralen Datenbank sind aktuell im Quellcode hinterlegt statt über Umgebungsvariablen oder eine externe Konfigurationsdatei bereitgestellt zu werden, was den Betrieb in unterschiedlichen Umgebungen (Entwicklung, Testbetrieb im Labor, produktiver Einsatz) erschwert.

Kapitel 6

Fazit und Ausblick

6.1 Zusammenfassung der Ergebnisse

Im Rahmen dieser Arbeit wurden drei zuvor unabhängig voneinander entwickelte Teilsysteme zur Wildtierforschung — ein Bärenabschreckungssystem, eine Bewegungssimulation sowie ein Bildextraktionssystem — zu einer gemeinsamen, Multi-Layer-fähigen Kartenplattform zusammengeführt. Zentrale technische Grundlage hierfür war der Wechsel der Kartenvisualisierung von der DOM-basierten Bibliothek Folium/Leaflet zur WebGL-basierten Bibliothek MapLibre GL JS, der eine deutlich performantere Darstellung großer, aus Simulation und GPS-Tracking resultierender Datenmengen ermöglichte. Ergänzend wurde die zuvor heterogene Datenhaltung (verteilte SQLite-Datenbanken, CSV-Dateien, unterschiedliche Tabellenschemata) in eine zentrale MariaDB-Datenbank konsolidiert und die Fehlerbehandlung der Datenzugriffsschicht so überarbeitet, dass Verbindungsprobleme für Nutzende nachvollziehbar sichtbar werden, anstatt zu stillen, schwer diagnostizierbaren Datenlücken zu führen. Durch gezieltes serverseitiges Caching sowie eine klare Trennung zwischen serverseitigen Filteroperationen und clientseitigen Darstellungsinteraktionen konnte zudem die wahrgenommene Reaktionsgeschwindigkeit der Anwendung spürbar verbessert werden.

6.2 Ausblick auf zukünftige Erweiterungen

Aufbauend auf dem in dieser Arbeit geschaffenen Fundament ergeben sich mehrere sinnvolle Erweiterungsrichtungen für zukünftige Arbeiten:

- **Robustere Infrastruktur:** Einführung eines Monitorings bzw. eines automatisierten Health-Checks für die zentrale Datenbank sowie ggf. eines Redundanz- oder Fallback-Konzepts, um die in Abschnitt 5.2 beschriebene Ausfallanfälligkeit zu reduzieren.

- **Natives Streamlit-Custom-Component:** Ablösung der aktuellen `iframe`-basierten Karteneinbindung durch ein bidirektionales Custom Component, das eine inkrementelle Datenübertragung sowie eine direkte Kommunikation zwischen Kartenzustand und Python-Backend ermöglicht, ohne bei jeder Interaktion die vollständige Karte neu aufzubauen.
- **Erweiterung auf weitere Arten und Regionen:** Die entwickelte Architektur ist bewusst generisch gehalten (gemeinsame Datenzugriffsschicht, generisches GeoJSON-Format) und ließe sich grundsätzlich auf weitere Tierarten oder Untersuchungsgebiete übertragen.
- **Echtzeit-Kollaborationsfunktionen:** Ergänzung von Konfliktlösungsmechanismen bzw. Live-Synchronisation für den Fall, dass mehrere Forschende gleichzeitig Markierungen oder Gebiete bearbeiten.

Literaturverzeichnis

[1] Lorenz, M.: *[Titel der Bachelorarbeit einfügen]*, OTH Regensburg, [Jahr].

[2] Breu, F.: *[Titel der Bachelorarbeit einfügen]*, [Jahr].

[3] Lelchat, A.: *[Titel der Arbeit einfügen]*, [Jahr].

[4] [Quelle zu GIS-Grundlagen / Geodaten einfügen]

[5] [Quelle zu WGS84/Web-Mercator einfügen]

[6] *The GeoJSON Format*, RFC 7946, IETF, 2016.

[7] [Quelle zu Vector Tiles einfügen]

[8] *Leaflet – an open-source JavaScript library for interactive maps*, offizielle Dokumentation, `https://leafletjs.com`.

[9] [Quelle zu Mapbox GL JS einfügen]

[10] *MapLibre GL JS*, GitHub-Repository, `https://github.com/maplibre/maplibre-gl-js`.

[11] *Mapbox Style Specification*, offizielle Dokumentation.

[12] [Quelle zu Step Selection Functions in der Bewegungsökologie einfügen, z. B. Fortin et al.]